

Assignment 2 (15 points)

1. Match the following set of bit operations in A to the outcomes shown in B. (2 points)

A. Bit operations

- i. $x \& (x - 1)$
- ii. $x | (x - 1)$
- iii. $x \& (x + 1)$
- iv. $x | (x + 1)$

B. Outcomes

- a. Turn off the trailing 1-bits in a word, producing x if none (e.g., 01010111 => 01010000)
- b. Turn off the rightmost 1-bit in a word, producing 0 if none (e.g., 01011000 => 01010000)
- c. Turn on the rightmost 0-bit in a word, producing all 1s if none (e.g., 10100111 => 10101111)
- d. Turn on the trailing 0-bits in a word, producing x if none (e.g., 10101000 => 10101111)

2. Match the following set of bit operations in A to the outcomes shown in B. (2 points)

A. Bit operations

- i. $\sim x \& (x - 1)$
- ii. $\sim x | (x - 1)$
- iii. $\sim x \& (x + 1)$
- iv. $\sim x | (x + 1)$

B. Outcomes

- a. Create a word with 0's at the position of the trailing 1's and 1's elsewhere, producing all 1's if none (e.g., 11010111 => 11111000)
- b. Create a word with a single 0-bit at the position of the rightmost 1-bit and 1's elsewhere, producing all 1's if none (e.g., 10101000 => 11110111)
- c. Create a word with 1's at the position of trailing 0's and 0's elsewhere, producing 0 if none (e.g., 01011000 => 00000111)
- d. Create a word with a single 1-bit at the position of the rightmost 0-bit, producing 0 if none (e.g., 10100111 => 00001000)

3. Try the following operations with each of the following 8-bit binary strings, assuming 8-bit results: $x = 00000000$, $x = 11111111$, $x = 01011000$ and describe the results of the operations. For arithmetic right shift, interpret the binary string as a signed two's-complement value. (3 points)

- a. $x \wedge (x - 1)$
- b. $x \wedge (x + 1)$
- c. $x \ll 2$
- d. $x \gg 2$ logical right shift
- e. $x \gg 2$ arithmetic right shift

Programming Section

Array manipulation

Write the following array functions. You can assume that these are integer arrays. Your function bodies should not use `[ ]` operators. Instead, use pointers to access array elements. Unless otherwise specified, manipulate the original array in place. Do not create an additional array.

For each function assigned below, suppose `arr[] = {1, 4, 6, 5, 2, 7, 10}` for the purposes of discussion.

Note: It is okay to use array notation in main to create and initialize arrays to test your functions.

- a. `int search(int arr[], int n, int val)` (2 points)

Behavior: Search for a given value in the array and return the index of the location. Return -1 if not found in the array.

Parameters:

`int arr[]`: The array to be searched.

`int n`: The size/length of the array. That is, the number of elements in the array.

`int val`: The element to be searched for in the array.

Example: Using the above array `arr`, `search(arr, 7, 6)` should return 2, and `search(arr, 7, 8)` should return -1.

- b. `void reverse(int arr[], int n)` (3 points)

Behavior: Reverse the contents of the array. Note: The elements of the array passed to your function should actually be modified so that they are reversed. You should *not* attempt to create a new array and return that, as implied by the void return type!

Parameters:

`int arr[]`: The array to be reversed

int n: The size/length of the array. That is, the number of elements in the array.

Example: Using the above array arr, reverse(arr, 7) should result in arr having its contents rearranged as follows: arr = {10, 7, 2, 5, 6, 4, 1}.

c . void oddFirst(int arr[], int n) (3 points)

Behavior: Rearrange the array such that all the odd elements in the array are in the beginning of the array. There are no requirements as to the order of the elements beyond this one. As in problem b (reverse), you must mutate the array given as argument.

Parameters:

int arr[]: The array to be rearranged.

int n: The size/length of the array. That is, the number of elements in the array.

Example: Using the above array arr, oddFirst(arr, 7) could result in arr having its contents rearranged as follows: {1, 5, 7, 4, 6, 2, 10}. Note that this is the exact order that I believe the simplest implementation will produce, but as mentioned in **Behavior**, any arrangement is correct as long as the odd elements come first.